

CS3611

计算机网络大作业

2026 Spring

题目四：基于空口帧特性识别的摄像头检测系统

第 10 组

小组成员

序号	姓名	学号
1	赵晨星	524030910233
2	姜亦轩	524030910231
3	马铭源	524030910011
4	张熙和	524030910219
5	祖浩然	524030910250

2026 年 6 月 18 日

1 项目概述

1.1 实验目标

本项目对应任务四“基于空口帧特性识别的摄像头检测系统”。目标是在不知道隐藏无线摄像头 MAC 地址的情况下，使用支持 Monitor 模式的无线网卡被动监听 802.11 空口帧，解析 Radiotap 与 MAC 层字段，构建设备级流量画像，并通过规则判定与机器学习模型输出可疑摄像头 MAC 排名。

本项目具体要求如下：第一，理解 802.11 空口帧、Radiotap 头和 MAC 地址字段在 ToDS/FromDS 不同组合下的含义；第二，掌握 Linux 下 Monitor 模式、信道锁定、tshark/Wireshark 抓包与 pcap 离线复现流程；第三，将帧长、方向、RSSI、速率、IAT、QoS、突发等时间序列统计量转化为 MAC 级设备画像；第四，设计可解释规则分数，并引入 SVM 与 Random Forest 集成模型进行摄像头 AI 识别。

1.2 完成情况总览

系统已实现从抓包、解析、特征提取、规则筛选、SVM/RF 训练评估到未知 pcap 候选 MAC 排序的端到端流程。项目使用 CSV/JSON 作为轻量本地结构化存储，包括 labels.csv、特征 CSV、规则预测 CSV、评估 JSON 和图表结果，满足本地检索与对比需求。

表 1: 功能完成情况

类别	功能点	完成状态
基础功能	空口帧获取：Monitor 模式、信道锁定、tshark 固定时长/定包抓取、pcap 保存	完成
	字段分析：Radiotap 与 802.11 MAC 字段解析，统一为 FrameInfo	完成
	判决依据：按 MAC/窗口聚合，构造 camera_score 与候选排序	完成
	验证评估：混淆矩阵、检出率、误报率、AUC、PCA、特征重要性图	完成
扩展功能	摄像头 AI 方法识别：SVM + Random Forest + soft voting 二分类检测	完成
	多 MAC 并行候选筛查：未知 pcap 枚举多个 MAC 并逐一聚合预测	部分完成
	摄像头定位：保留 RSSI 均值、范围和趋势特征，未形成完整定位系统	未作为主线
	自定义开放问题：未单独扩展	未完成

目前标注文件 data/labels.csv 中包含 5 个真实设备 MAC，其中无线摄像头 2 个、平板 2 个、手机 1 个。当前主结果采用 10 秒 MAC 级设备窗口；flow 级外部复测保留为压力测试，用于暴露单 flow 泛化不足的问题。

2 系统设计

2.1 整体架构

系统采用“采集层－解析层－特征层－模型层－输出层”的分层设计。原始输入是 Monitor 模式下抓取的 802.11 pcap，解析层将不同来源的 Radiotap/MAC 字段统一为结构化帧对象；特征层同时提供 frame、flow 和 device-window 三个粒度，其中最终主线是以目标 MAC 为中心的 10 秒设备窗口；模型层包含可解释规则基线与 SVM/RF 集成检测器；输出层生成可疑 MAC 排名、指标表和可视化图。

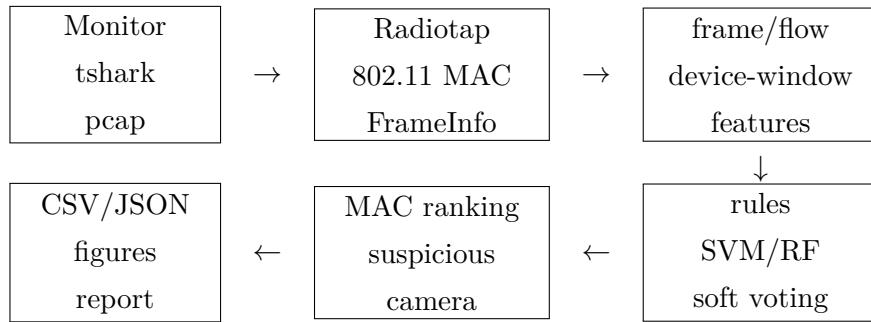


图 1: 端到端系统架构与数据流

主要代码目录与职责如下：`src/capture/` 负责网卡与抓包封装，`src/parser/` 负责 Radiotap/MAC 字段解析，`src/features/` 负责三层特征与突发检测，`src/ml/` 负责规则、SVM、RF、集成和模型持久化，`src/visualization/` 负责混淆矩阵、ROC/PR、PCA、特征重要性等图表。脚本入口集中在 `scripts/`，典型流程为 `collect/extract/train/evaluate/detect`。

2.2 协议/算法设计

无线摄像头的可识别性来自持续视频上传行为：相对长时间的 STA 到 AP 上行、较高大帧比例、较稳定的帧间隔、QoS Data 帧、规律突发以及较高吞吐量。系统不依赖 IP 负载内容，而是只使用 802.11 空口侧可观察到的 MAC 层与物理层统计特征。

表 2: 关键字段与特征来源

来源	字段/特征	作用
Radiotap	RSSI、data rate、channel、MCS	反映信号强度、物理层速率和信道环境
802.11 MAC	SA/DA/TA/RA/BSSID、ToDS/FromDS	判断有效源/目的地、STA→AP 上行与 AP→STA 下行
Frame	frame length、type/subtype、QoS、retry、protected	区分数据/管理/控制帧，识别 QoS Data 与大帧
Time series	IAT、burst count、burst density、throughput	描述连续上传、突发规律性和传输强度

规则基线采用加权分数：

$$\begin{aligned} score(x) = & 2\mathbb{I}(tx_packet_ratio \geq 0.6) + 2\mathbb{I}(uplink_packet_ratio \geq 0.5) \\ & + 2\mathbb{I}(large_frame_ratio \geq 0.4) + \mathbb{I}(qos_data_ratio \geq 0.4) \\ & + \mathbb{I}(cv_iat \leq 0.8) + \mathbb{I}(burst_count \geq 3) + \mathbb{I}(throughput_bps \geq 10^6). \end{aligned}$$

默认阈值为 5 分。前三项权重最高，因为它们分别对应“设备主要在发包”“主要是 STA 到 AP 上行”“大帧比例高”，是视频上传流最核心的证据；QoS、IAT 稳定、突发和吞吐量作为辅助证据。

AI 检测器以数值特征矩阵为输入，先删除 `device_mac`、`device_oui`、`source_file`、`session_id`、窗口编号和启发式分数等高风险字段，避免模型记住设备身份；随后使用 `StandardScaler` 标准化，分别训练 SVM 和 Random Forest，最后用 soft voting 融合两个模型的概率。未知 pcap 检测时，脚本先枚举 MAC，再对每个候选 MAC 提取 10 秒设备窗口特征，汇总 `camera_prob_mean` 和 `camera_window_ratio`，默认任一指标超过阈值即标记为 `suspicious_camera`。

2.3 模块划分与分工

小组分工按照任务要求与最终 Poster 口径整理如下。

表 3: 模块分工

模块	职责描述	负责人
主链路与抓包处理	大部分代码实现与整合，Monitor 抓包、pcap 处理、端到端流程串联与最终上台展示	赵晨星
AI 分类与检测算法	摄像头 AI 方法识别，SVM/RF/soft voting、规则解释、未知 pcap 候选概率汇总	姜亦轩
协议解析与特征抽取	Radiotap/MAC 字段解析，FrameInfo 结构，frame/flow/device-window 特征抽取与分析	马铭源
测试评估与可视化	综合测试验证，混淆矩阵、ROC/PR、PCA、特征重要性等结果图和指标解释	张熙和
文档与交付材料	README、操作说明、组内说明文档、测试整理、数据结果整理与提交准备	祖浩然

3 核心实现

3.1 Monitor 抓包与离线 pcap 输入

抓包层使用 `tshark` 作为核心工具。系统先将无线网卡切换到 Monitor 模式并锁定目标 AP 所在信道，再支持按固定时长或固定包数保存 pcap。离线 pcap 是后续解析、特征提取和复现实验的统一输入，这也避免现场无线环境不稳定时影响报告评估。

关键实现位于 `src/capture/capture_session.py`。定时抓包通过 `tshark -i interface -a duration:T -w output.pcap` 完成，若指定信道则先调用 `set_channel()` 锁定信道。该设计保证每次采集都有确定时长、确定信道和可复现的 pcap 文件。

Listing 1: 固定时长抓包核心逻辑

```

1 cmd = ['tshark', '-i', self.interface,
2       '-a', f'duration:{duration_sec}',
3       '-w', output_path]
4 if channel is not None:
5     set_channel(self.interface, channel)
6 subprocess.run(cmd, check=True)

```

实际应用中, 由于 `tshark` 命令在 `sudo` 模式下无法使用, 因而使用 `bash` 脚本完成模式切换、锁定信道并最后开始抓包等操作。

3.2 Radiotap/MAC 解析与地址方向判定

802.11 帧中地址字段的含义依赖 ToDS/FromDS, 不能简单把某个字段当成源地址。项目在 `src/parser/pcap_reader.py` 中定义 `FrameInfo`, 统一保存时间戳、帧长、RSSI、速率、信道、帧类型、ToDS/FromDS、SA/DA/TA/RA/BSSID、QoS 和重传标志。对于摄像头检测, 最重要的方向判断是 `to_ds=1`, `from_ds=0`, 即 STA 到 AP 上行。

Listing 2: STA 到 AP 上行判断

```

1 @property
2 def is_uplink(self):
3     return self.to_ds == 1 and self.from_ds == 0

```

Radiotap 层提供 RSSI、data rate、channel frequency、MCS 等物理层信息; MAC 层提供帧类型、QoS、地址和方向信息。解析失败或驱动缺失字段时, 代码使用 `None/0/NaN` 做降级处理, 保证后续 `DataFrame` 不因单个字段缺失而中断。

3.3 MAC 设备窗口画像与突发特征

早期 flow 粒度按 SA→DA 聚合, 但单条 flow 容易被会话切分, 也容易受到手机视频通话、短时上传等干扰。最终主结果采用 device-window 粒度: 以某个目标 MAC 为中心, 收集该 MAC 作为源或目的地址出现的所有帧, 并按 10 秒窗口聚合。每个窗口统计包量、字节数、吞吐量、帧长分布、大帧比例、IAT 均值/方差/CV、tx/rx 比例、uplink/downlink 比例、RSSI、速率、帧类型比例、QoS 比例、重传比例、突发数量与突发密度。

Listing 3: MAC 设备窗口聚合核心逻辑

```

1 frames = [f for f in frame_features
2           if normalize_mac(f.get('sa')) == target_mac
3           or normalize_mac(f.get('da')) == target_mac]
4 win_idx = int((frame['timestamp'] - t0) / window_sec)
5 rows.append(extract_device_features(win_frames, target_mac))

```

突发检测位于 `src/features/burst_detector.py`。其思想是: 若相邻帧 IAT 小于 1 ms, 则视为同一突发中的连续帧; 连续帧数不少于 3 时记录一个 burst。摄像头视频流通常会表现出较多连续上传突发和较大的突发字节数, 因此 `burst_count`、`mean_burst_bytes`、`burst_density` 等特征进入规则与模型。

3.4 规则基线与 AI 集成检测

规则基线位于 `src/ml/rule_baseline.py`，用于给出可解释的候选分数。规则不是只看一个字段，而是将“持续上行视频流”拆成多条证据：发送占比、上行占比、大帧占比、QoS、IAT 稳定性、突发和吞吐量。默认阈值为 5 分，输出 `rule_score` 与触发的规则名称。

Listing 4: 规则分数计算

```
1 for rule in rules:
2     if _rule_matches(row, rule):
3         score += rule['weight']
4         triggered.append(rule['name'])
5 predictions = (scored['rule_score'] >= threshold).astype(int)
```

AI 模型位于 `src/ml/`。SVM 使用 RBF/poly kernel 与 GridSearchCV 调参，适合标准化后的中小规模多维特征；Random Forest 使用树模型捕捉非线性组合，并提供 Gini 特征重要性；soft voting 将 SVM 和 RF 的概率平均融合，输出更平滑的摄像头概率。

Listing 5: 训练前删除身份泄露字段

```
1 id_cols = ['sa', 'da', 'sa_oui', 'source_file', 'session_id',
2           'device_mac', 'device_oui', 'window_idx', 'window_start',
3           'is_known_camera_oui', 'camera_heuristic_score', label_col]
4 X_df = feature_df.drop(columns=id_cols, errors='ignore')
```

这一处理很关键：如果保留 `device_mac`、OUI 或 `source_file`，模型可能只记住“哪台设备来自哪次采集”，而不是学习摄像头流量行为。删除这些字段后，模型主要依赖帧长、方向、IAT、突发、速率、吞吐量等可泛化特征。

3.5 未知 pcap 的候选 MAC 排序

演示入口为 `scripts/detect_unknown_pcap.py`。它不需要 `labels.csv`，先用 tshark 枚举 pcap 中出现的 MAC，并按 `data_source_frames`、`source_frames`、`total_frames` 选择候选设备；再逐个 MAC 抽取 10 秒 device-window 特征，调用训练好的摄像头检测器；最后聚合窗口级预测，输出 `camera_prob_mean`、`camera_prob_max`、`camera_window_ratio` 和 `suspicious_camera`。

该流程直接对应任务要求中的“对环境中的所有 MAC 的流量画像进行筛选与排序，输出可疑视频上传流对应的候选 MAC 地址及关键指标”。输出 CSV 可作为现场演示和离线复核材料。

4 测试与结果

4.1 测试环境

项目要求的采集环境为 Linux/Ubuntu、支持 Monitor 模式的外置 WiFi 网卡、摄像头/手机/平板等终端、Wireshark/tshark 与 aircrack-ng。代码依赖见 `requirements.txt`，主要为 `numpy`、`pandas`、`scikit-learn`、`pyshark`、`scapy`、`matplotlib`、`seaborn`、`joblib` 和 `tqdm`。由于训练代码使用 `StratifiedGroupKFold`，完整 ML 训练评估环境建议满足 `scikit-learn` 1.3.0 及以上版本。

- **采集工具**: tshark/Wireshark、aircrack-ng、支持 Monitor 模式的无线网卡。
- **分析语言**: Python 3, 项目依赖版本以 `requirements.txt` 为准。
- **数据输入**: Monitor 模式采集的 802.11 pcap/pcapng 文件。
- **标注数据**: 5 个标注 MAC, 其中 2 个 wireless_camera、2 个 tablet、1 个 smartphone。

4.2 功能测试

功能测试主要覆盖从 pcap 到可疑 MAC 排名的完整链路。仓库中的 `tests/` 覆盖 PcapReader、Radiotap 解析、突发检测、帧/流特征、ML 数据集、SVM/RF/ensemble、模型评估与持久化等模块; 最终展示结果以 `report/final_device_window_test/`、`report/final_test/` 和 `report/rule_baseline_test/` 下的图表与 JSON/CSV 为准。

表 4: 主要评估结果汇总

评估项	AUC	检出率	误报率	Precision	F1	说明
设备窗口级主结果	1.000	100.0%	0.0%	100.0%	100.0%	10 秒 MAC 窗口, TP=36、FP=0、TN=54、FN=0
规则基线 10 秒窗口	—	100.0%	0.0%	100.0%	100.0%	camera_score 阈值 5, 结果与主窗口评估一致
规则基线 1 秒窗口	—	100.0%	0.22%	99.72%	99.86%	TP=360、FP=1、TN=444、FN=0, 说明短窗口下仅 1 个误报
flow 级外部复测	0.511	83.5%	85.7%	63.1%	71.9%	TP=329、FP=192、TN=32、FN=65, 用作泛化压力测试

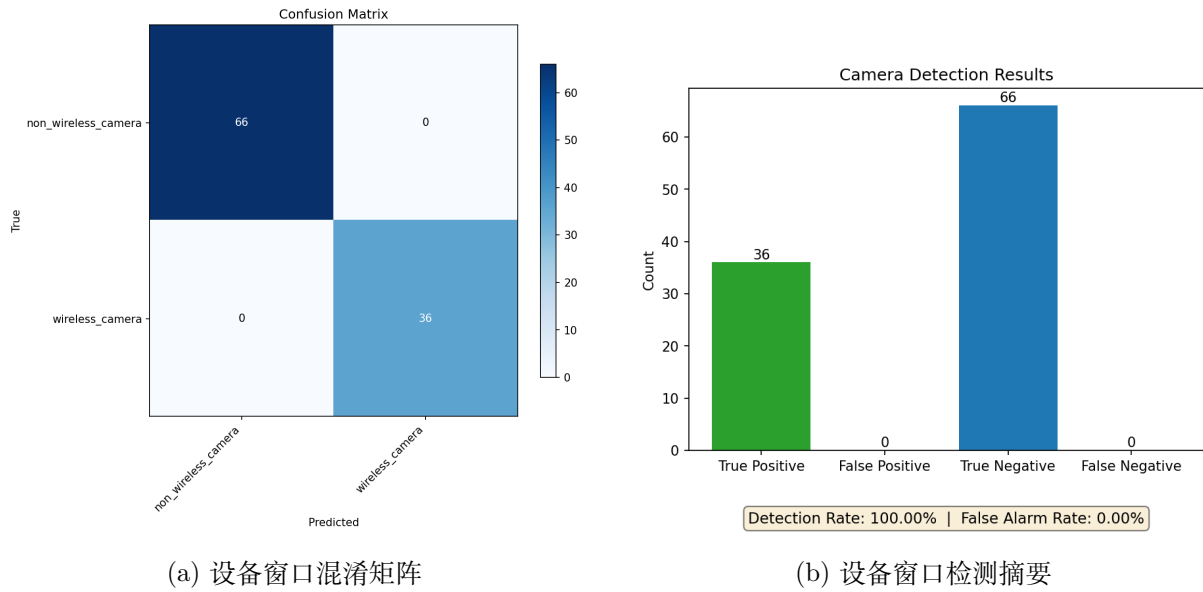


图 2: 主结果: MAC 级 10 秒设备窗口评估

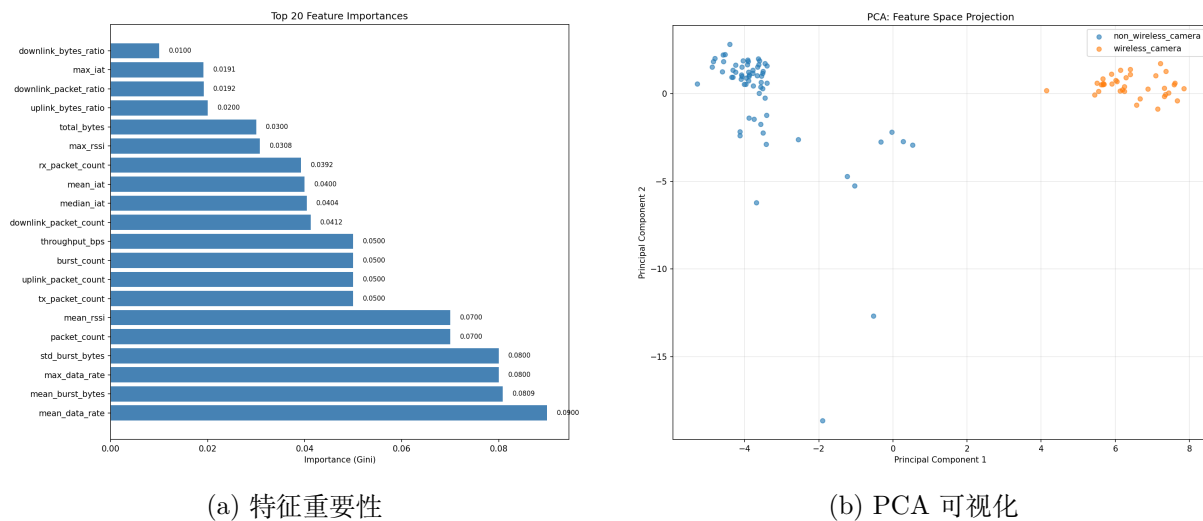


图 3: 设备窗口特征解释

4.3 结果分析

主结果表明, 按 MAC 聚合后的设备窗口更符合“找摄像头 MAC”这一任务目标。当前标注样本中, 摄像头窗口与非摄像头窗口在大帧比例、包量、突发字节数、数据速率、IAT 统计和上行相关特征上可分性较强, 因此设备窗口级 AUC 达到 1.000, 且无误报、无漏报。

特征重要性图中靠前的特征包括 `large_frame_ratio`、`packet_count`、`min_frame_size`、`mean_burst_bytes`、`mean_data_rate`、`max_iat`、`std_iat`、`uplink_packet_count` 和 `burst_count`。这些特征与摄像头持续上传视频的行为一致: 视频流会产生较多数据帧、较高大帧比例、较明显的突发字节数以及相对稳定的发送节奏。由于训练中删除了 MAC/OUI/source file 等身份字段, 模型结果更能反映流量行为而非设备记忆。

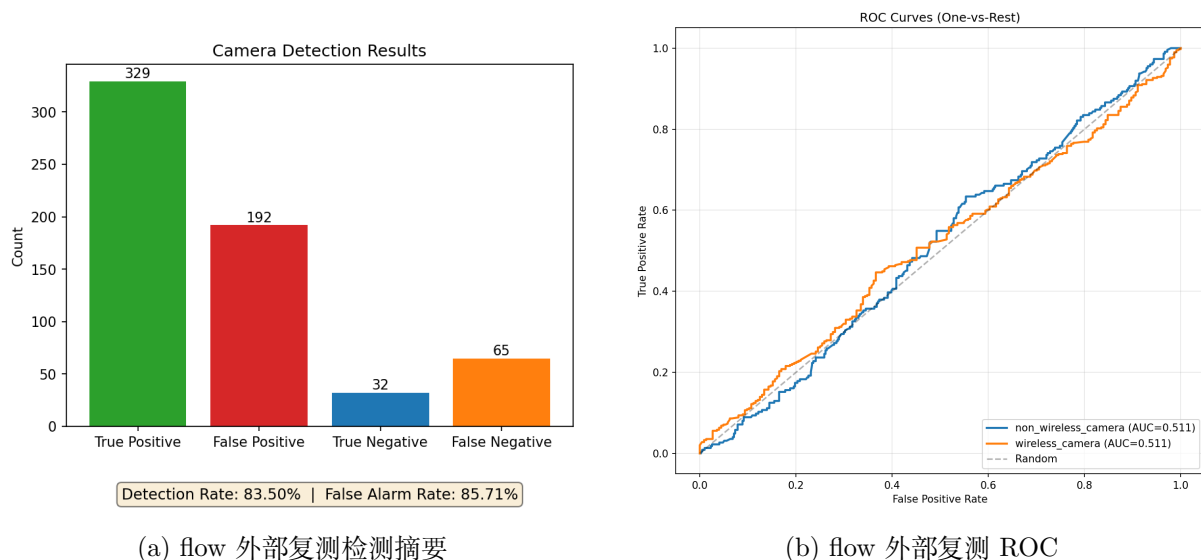


图 4: 压力测试: flow 粒度外部复测

flow 级外部复测 AUC 为 0.511, 误报率为 85.7%。这个结果没有被作为主结论, 而是用于说明单 flow 粒度的局限: flow 只是 SA→DA 的碎片化通信片段, 跨场景时普通手机视频通话、直播推流、云盘上传、AP 管理帧等 hard negatives 也会呈现大帧、上行和突发特征, 导致模型误报。后续需要补充更多 hard negatives, 并按房间、日期、设备做 group split, 进一步验证泛化能力。

5 总结与心得

本项目完成了隐藏无线摄像头检测的端到端原型: 从 Monitor 模式抓取 802.11 空口帧, 到 Radiotap/MAC 字段解析, 再到 frame、flow、device-window 多层特征工程, 最终通过规则分数和 SVM/RF 集成模型输出可疑摄像头 MAC。设备窗口级主结果达到 AUC 1.000、检出率 100%、误报率 0%, 说明在当前标注样本中, 基于 MAC 级时间窗口的设备画像可以有效区分摄像头与普通终端。

项目中最大的难点是 802.11 地址字段和方向判断。与普通以太网不同, 802.11 的 SA、DA、TA、RA、BSSID 语义会随 ToDS/FromDS 改变, 若解析错误, 就会直接影响上行比例、设备聚合和候选 MAC 输出。另一个难点是泛化评估: flow 级外部复测暴露出普通视频通话和短时上传容易造成误报, 因此最终报告将 device-window 作为主线, 同时保留 flow 压力测试作为局限性分析。

通过本项目, 小组对计算机网络中的链路层帧结构、无线信道监听、抓包工具、时间序列流量画像和网络行为分类有了更完整的理解。后续改进方向包括: 扩大真实摄像头与非摄像头设备数据集, 补充手机视频通话、会议投屏、网盘上传等 hard negatives; 严格按设备/场景/日期做分组验证; 进一步调节 camera_threshold 与 window_ratio_threshold; 结合更长时间窗口和 RSSI 变化趋势, 探索摄像头空间定位能力。

6 附录：提交文件清单与使用手册

请在下表中列出随本报告一起打包提交的所有文件，并简要说明用途与使用方式。助教将根据此表检查提交内容的完整性。

表 5: 提交文件清单 (示例)

序号	文件/目录名	说明
1	report.pdf	项目报告（即本文档）
2	src/	项目完整源代码
3	demo_video.mp4	功能演示录屏
4	scripts/	脚本文件，使用方式见下方说明
5	requirements.txt	Python 依赖列表
6	README.md	项目

环境配置与启动说明：

本项目使用的虚拟机环境配置参数如下：

Listing 6: 虚拟机配置

```

1 VMware: Workstation Pro / Player
2 Ubuntu: 22.04 LTS Desktop
3 Disk: 30 GB
4 Memory: 4-8 GB
5 CPU: 2-4 cores
6 Network Adapter: NAT
7 USB Controller: USB 3.0 / USB 3.1
8 Installation: Minimal installation

```

安装好虚拟机后，下载工具：

Listing 7: 工具包

```

1 sudo apt update
2 sudo apt install -y python3 python3-pip git tshark aircrack-ng wireless-tools net-
  tools

```

安装 ‘tshark’ 时如果提示：

Listing 8: tshark 安装提示

```

1 Should non-superusers be able to capture packets?

```

可以选择 **Yes**。不过本实验抓 802.11 空口帧时通常仍建议使用 **sudo**。如果选择了 **Yes**，则后续使用 **bash** 脚本而非 **python** 脚本。

之后连接网卡按照如下步骤即可开始抓包以及检测：

Listing 9: 启动步骤示例

```
1 # 1. 安装依赖
2 pip install -r requirements.txt
3
4 # 2. 获取网卡名
5 sudo airmon-ng check kill
6 sudo airmon-ng start wlan0
7
8 iwconfig
9
10 假设网卡名叫 wlan0
11
12 # 3. 启动抓包脚本
13 bash scripts/capture_monitor.sh \
14     -i wlan0 \
15     -o data/raw/test_capture.pcap \
16     -t 60 \
17     -c 6
18
19 或者:
20 sudo python3 scripts/collect_training_data.py \
21     -i wlan0mon \
22     -d wireless_camera \
23     -t 60 \
24     -c 6 \
25     -m aa:bb:cc:dd:ee:ff \
26     -n "camera_live_view"
27
28 # 3. 启动检测脚本
29 python3 scripts/detect_unknown_pcap.py -p data/raw/test_capture.pcap --method both
    -m data/models --window 10 --top-macs 20 -o data/processed/
    unknown_scene_detection.csv
```

更多启动脚本使用，如训练、特征提取等见 README。